Introdução à Computação
busca linear vs busca binária

Alexandre Rademaker

Busca linear I

Queremos localizar um valor em um array. O computador só consegue examinar uma posição de memória por vez!

Se nada pudermos assumir sobre a ordem dos valores no array, temos que olhar posição por posição. Buscando por 0.

```
For each position from left to right
  If number is equal 0
    Return true
Return false
```

Busca linear é $O(n)$ e $\Theta(n)$. Por que?

Busca linear II

src/numbers1.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef char* string;

int main(int argc, string argv[]) {
  int numbers[] = {4, 6, 8, 2, 7, 5, 0};

  if(argc < 2) {
    printf("Usage: %s NUMBER\n", argv[0]);
    return 1;
  }

  int t = atoi(argv[1]);

  for (int i = 0; i < 7; i++) {
    if (numbers[i] == t) {
      printf("Found at %d\n", i);
      return 0;
    }
```

Busca linear III

```
21    }
22    printf("Not found\n");
23    return 1;
24 }
```

Busca linear IV

src/names0.c

```
int main() {
  string names[] =
    {"Bill", "Charlie", "Fred",
     "George", "Ginny", "Percy", "Ron"};

   for (int i = 0; i < 7; i++) {
     if (names[i] == "Ron") {  // BUG
       printf("Found\n");
       return 0;
     }
   }
   printf("Not found\n");
   return 1;
 }
```

Busca linear V

src/names1.c

```
int main() {
  string names[] =
    {"Bill", "Charlie", "Fred",
     "George", "Ginny", "Percy", "Ron"};

   for (int i = 0; i < 7; i++) {
     if (strcmp(names[i], "Ron") == 0) {
       printf("Found\n");
       return 0;
     }
   }
   printf("Not found\n");
   return 1;
 }
```

Busca linear VI

atenção Usando o código abaixo teríamos o oposto do que queremos, pois valores diferentes de zero seriam considerados 'true'. Vide strcmp.

```
1 if(strcmp(names[i], "Ron")) { ... }
```

Usando a negação funcionaria neste caso, mas seria um código pior, pois não verifica explicitamente o valor 0, que desejamos.

```
1 if(!strcmp(names[i], "Ron")) { ... }
```

Busca binária I

Queremos localizar um valor em um array. O computador só consegue examinar uma posição de memória por vez!

Se pudermos assumir que os valores estão **ordenados**, podemos usar a busca binária. Buscando por N.

```
If no position
 Return false
If number in middle position equals N
 Return true
Else if N < number in the middle position
 Search left half
Else if N > number in the middle position
 Search right half
```

Busca binária é $O(\log n)$ e $\Omega(1)$. Por que?

Busca binária II

algumas considerações . . .

É muito mais eficiente que a busca linear, mas requer que o array esteja **ordenado**.

Podemos ordenar o array antes. . . Mas isso também irá demandar tempo. Se várias buscas forem ser executadas, a ordenação pode valer à pena (bancos de dados).

Busca Binária Interativa em C I

src/numbers2.c

```
1  int binary_search(int a[], int n, int x) {
2    int low = 0, high = n - 1;
3    while (low <= high) {
4      int k = (high + low) / 2;
5      if      (a[k] == x) return k;
6      else if (a[k] <  x) low  = k + 1;
7      else                high = k - 1;
8    }
9    return -1;
10 }
11
12 int main(int argc, string argv[]) {
13   int q, a[] = {-31, 0, 1, 2, 2, 4, 65, 83, 99, 782};
14
15   if (argc > 1)  q = atoi(argv[1]);
16   else           return 1;
17
18   int n = sizeof(a)/sizeof(a[0]), i = binary_search(a, n, q);
19   if (i >= 0) {
20     printf("%d is at index %d.\n", q, i);
21   }
22   else {
23     printf("%d is not found.\n", q);
24   }
25   return 0;
26 }
```

Busca Binária Interativa em C II

v	-31	0	1	2	2	4	65	83	99	782
p	0	1	2	3	4	5	6	7	8	9

Buscando por 99, números são posições do array

low	high	k	n
0	9	4	10
5	9	7	10
8	9	8	10

Observações I

Mas o código C mostrado é **interativo**, bem diferente do pseudo-código que é **recursivo**!

A stdlib.h onde achamos a função atoi.

O operador sizeof de C.

Podemos **instrumentar** o código para entender seu funcionamento?

Observações II

Podemos prever este tipo de erro? Como preparar o programa para um usuário menos **colaborativo**?

```
> ./numbers2 hello
0 is at index 1.
```